Procedure and Data Flow for a Backtest Session

Contents

1 Procedure and data flow for a backtest session **2**

1.1 Session setup (`BacktestTradingSession`) . . . 2
1.2 Event-driven timeline (per business day) . . . 2
1.2.1 How the rebalance schedule is generated . . . 2
1.3 Core loop behavior by event . . . 3
1.3.1 What `broker.update(dt)` does behind the scenes . . . 3
1.3.2 What `signals.update(dt)` does behind the scenes . . . 4
1.3.3 What `qts(dt, stats=stats)` does behind the scenes . . . 5
1.3.4 Where `submit_order(...)` is called in the backtest workflow . . . 5
1.3.5 Where open orders are checked and executed . . . 6
1.4 Rebalance pipeline (`PortfolioConstructionModel`) . . . 6
1.5 Broker execution and accounting path . . . 7
1.5.1 Exchange gate and execution lifecycle details . . . 7
1.6 End products from `run()` . . . 7
1.7 Sequence diagram (compact) . . . 8
1.8 Portfolio construction flowchart (tiny) . . . 8

2 Strategy logic (using `TopNMomentumAlphaModel` as an example) **8**

1 Procedure and data flow for a backtest session

This consolidates the execution path documented in `docs/trading.md`, `docs/simulation.md`, `docs/portcon.md`, and `docs/broker.md`.

1.1 Session setup (`BacktestTradingSession`)

At construction time, the session wires together:

- `DailyBusinessDaySimulationEngine` to generate a time-ordered stream of events.
- `SimulatedBroker` (with `Exchange`, `BacktestDataHandler`, `FeeModel`) to price and execute orders.
- `QuantTradingSystem` (`AlphaModel` + optional `RiskModel` + `PortfolioConstructionModel`) to create rebalance orders.
- Rebalance schedule (`daily`, `weekly`, `end_of_month`, or `buy_and_hold`) and optional `SignalsCollection`.
- Cash/account bootstrap via broker account + portfolio funding calls.

1.2 Event-driven timeline (per business day)

`DailyBusinessDaySimulationEngine` can yield four ordered events:

1. `pre_market` (00:00 UTC)
2. `market_open` (14:30 UTC)
3. `market_close` (21:00 UTC)
4. `post_market` (23:59 UTC)

In the default `BacktestTradingSession` wiring, the simulation engine is created with `pre_market=False` and `post_market=False`, so the active loop events are typically `market_open` and `market_close` only.

1.2.1 How the rebalance schedule is generated

The rebalance schedule is **precomputed once** during `BacktestTradingSession` construction and stored as `self.rebalance_schedule`, a `list[pd.Timestamp]`.

The creation flow is:

1. `BacktestTradingSession.__init__` stores the user-selected `rebalance` mode (such as `buy_and_hold`, `daily`, `weekly`, or `end_of_month`).
2. It calls `_create_rebalance_event_times()` to build the full schedule for the backtest date range.
3. That helper instantiates one concrete rebalance class and returns its `.rebalances` list.
4. Later, the run loop checks `is_rebalance_event(dt)` by testing whether the current event timestamp is a member of that precomputed list.

The supported schedule generators are:

- `BuyAndHoldRebalance` — one rebalance at the first eligible business-day timestamp near `start_dt`.
- `DailyRebalance` — every business day at the rebalance timestamp used by the simulation.
- `WeeklyRebalance` — a specified weekday (for example `WED`) across the backtest range; this mode requires `rebalance_weekday`.
- `EndOfMonthRebalance` — the last business day of each month across the backtest range.

Operationally, this means rebalance timing is **not recalculated on every loop iteration**. Instead, the session computes all rebalance datetimes up front, then uses a simple membership check to decide when `qts(dt, stats=stats)` should run.

1.3 Core loop behavior by event

```
for event in simulation_engine:
    dt = event.ts

    # Always refresh portfolio pricing; execute queued orders only if exchange is open.
    broker.update(dt)

    if event_type == "market_close":
        signals.update(dt)                          # optional
        if dt >= burn_in_dt:
            equity_curve.append((dt, broker.get_account_total_equity()))

    if is_rebalance_event(dt) and dt >= burn_in_dt:  # rebalance condition: dt is in
    precomputed rebalance_schedule
        qts(dt, stats=stats)                        # execute rebalance; triggers portfolio
    construction + order submission
```

1.3.1 What `broker.update(dt)` does behind the scenes

Although it appears as a single call in the session loop, `broker.update(dt)` performs several state transitions:

1. **Advance broker time** — The broker moves its internal clock to `dt`. Time is expected to be monotonic; backward moves are invalid.
2. **Mark every held position to market** — For each portfolio and each currently-held asset, the broker asks `BacktestDataHandler` for the latest **mid price** at `dt`, then updates the portfolio's stored market value for that asset. This happens even when the exchange is closed.
3. **Check whether the exchange is open** — The broker calls `exchange.is_open_at_datetime(dt)`. If the answer is false, no queued orders are executed and the update ends after mark-to-market.
4. **Drain queued orders when open** — If the exchange is open, queued orders are pulled from broker-managed order queues across portfolios.
5. **Sort execution order by direction** — Sell/short orders are executed before buy orders so that cash can be freed before subsequent purchases.

6. **Price each order using bid/ask** — For every queued order, the broker fetches the latest `(bid, ask)` pair from `BacktestDataHandler`; buys fill at the **ask**, sells fill at the **bid**.

7. **Estimate transaction cost** — The broker calculates trade consideration from execution price and quantity, then asks `FeeModel.calc_total_cost(...)` for commissions/fees.

8. **Create a `Transaction` and mutate portfolio state** — The broker builds a `Transaction(...)` object and passes it to `Portfolio.transact_asset(...)`, which updates the `Position`, adjusts cash, records a `PortfolioEvent`, and refreshes aggregate portfolio totals such as market value, equity, and PnL.

9. **Leave any non-executable orders queued until a later open event** — Orders submitted when the venue is closed (commonly at `market_close` in the default session) persist until a later `broker.update(dt)` occurs while the exchange is open.

In short, `broker.update(dt)` has a dual role: it is both the **mark-to-market valuation step** for all existing holdings and the **execution engine entry point** for any orders already waiting in the broker queue.

1.3.2 What `signals.update(dt)` does behind the scenes

At `market_close`, `signals.update(dt)` refreshes the internal state that alpha and risk models read later during rebalancing. Although it appears as one call, it performs a coordinated rolling-data update:

1. **Refresh each signal's active asset set** — For every managed `Signal`, the collection asks the associated universe for `universe.get_assets(dt)`. Newly-eligible assets are added, while assets that are no longer in the current universe are removed from the signal's tracked state.

2. **Fetch the latest market price for each tracked asset** — For each signal/asset pair, the signal layer requests the latest **mid price** from `BacktestDataHandler` at `dt`.

3. **Append prices into rolling buffers** — Each fetched price is pushed into that signal's `AssetPriceBuffers`. Internally, buffers are fixed-length deques keyed by asset and lookback period, so older observations fall off automatically once the buffer is full.

4. **Maintain lookback-specific history** — Simple indicators keep exactly `lookback` prices, while return/volatility-style indicators may internally keep `lookback + 1` prices so that period-to-period changes can be computed correctly.

5. **Advance signal warmup state** — After all prices are appended, the collection increments its warmup counter, which allows downstream models to determine whether enough history has accumulated to trust a signal value.

6. **Expose updated signal values to later stages** — After the update completes, alpha models and risk models can query the collection/signals for current indicator values derived from the refreshed buffers.

In short, `signals.update(dt)` is the **rolling market-data ingestion step** for the research layer: it keeps each signal's asset membership, lookback history, and warmup state synchronized with the latest close so the next rebalance decision uses up-to-date indicators.

1.3.3 What `qts(dt, stats=stats)` does behind the scenes

When the rebalance-event check passes, `qts(dt, stats=stats)` invokes the `QuantTradingSystem` and performs the full **decision-to-order-submission** pipeline:

1. **Enter `QuantTradingSystem.__call__`** — The system acts as the orchestration layer for rebalance-time logic. Its job is to ask portfolio construction for rebalance orders and then pass those orders to the execution layer.

2. **Run portfolio construction** — `PortfolioConstructionModel.__call__(dt, stats=stats)` generates target portfolio intent from the current market/signal state.

3. **Generate target weights** — The portfolio construction model calls `AlphaModel(dt)` to obtain raw weights, optionally passes them through `RiskModel(dt, weights)`, and then applies the configured `PortfolioOptimiser`.

4. **Expand weights to a full tradable set** — Current holdings are unioned with current universe assets so that assets already held but no longer desired receive target weight `0.0` and can be liquidated.

5. **Persist target allocations into `stats`** — Before sizing orders, the portfolio construction layer appends a snapshot of the target allocation map for `dt` into `stats['target_allocations']`. This is what later becomes `BacktestTradingSession.target_allocations`.

6. **Size target positions** — The `OrderSizer` converts target weights into integer share quantities using broker/account state, current prices, and any long-only cash buffer or long/short leverage rules.

7. **Compare target vs current holdings** — The model queries `broker.get_portfolio_as_dict(...)`, computes quantity deltas for each asset, and creates a `list[Order]` containing only non-zero rebalance trades.

8. **Hand orders to execution** — `ExecutionHandler.__call__(dt, rebalance_orders)` applies the configured `ExecutionAlgorithm` and, in the default backtest setup, submits the resulting orders to the broker.

9. **Trigger broker-side queueing/execution** — With `submit_orders=True`, each submitted order is passed to `broker.submit_order(...)`, followed by `broker.update(dt)`. Whether those orders execute immediately or remain queued depends on the exchange-open check described in §3.1 and §5.1.

In short, `qts(dt, stats=stats)` is the **rebalance orchestration step**: it turns updated signals and current portfolio state into target weights, sized positions, submitted orders, and a recorded snapshot of target allocations.

1.3.4 Where `submit_order(...)` is called in the backtest workflow

`submit_order(...)` is not called directly from `BacktestTradingSession.run()`. It is called indirectly through the quant system execution chain:

1. The run loop hits a rebalance event and calls `qts(dt, stats=stats)`.

2. `QuantTradingSystem.__call__(...)` generates `rebalance_orders` via portfolio construction.

3. `ExecutionHandler.__call__(dt, rebalance_orders)` iterates final orders.

4. For each order (when `submit_orders=True`), it calls `broker.submit_order(self.broker_portfolio_id, order)` and then `broker.update(dt)`.

So the concrete submit path is:

`BacktestTradingSession.run` → `qts(...)` → `ExecutionHandler.__call__(...)` → `broker.submit_order(...)`

After submission, orders enter the broker queue and are executed immediately only if the exchange-open check in `broker.update(dt)` passes.

1.3.5 Where open orders are checked and executed

Open orders are checked/executed inside `SimulatedBroker.update(dt)`, which is called from two places in the backtest flow:

1. **Every simulation event** in `BacktestTradingSession.run()` via `self.broker.update(dt)`.

2. **Immediately after each submitted order** in `ExecutionHandler.__call__` via `self.broker.update(dt)`.

Within `SimulatedBroker.update(dt)`, the execution path is:

1. Check venue state with `exchange.is_open_at_datetime(self.current_dt)`.

2. If open, drain per-portfolio order queues from `self.open_orders[...]`.

3. Sort drained orders by `order.direction` (short/sell first, then long/buy).

4. Execute each order via `_execute_order(dt, portfolio_id, order)`.

If the exchange is closed, orders remain queued in `self.open_orders` and are retried on a later `broker.update(dt)` call when the exchange is open.

1.4 Rebalance pipeline (`PortfolioConstructionModel`)

When rebalancing is triggered, data flows through these stages:

1. `AlphaModel(dt)` → raw `weights: dict[asset, float]`

2. `RiskModel(dt, weights)` → risk-adjusted weights (optional)

3. `PortfolioOptimiser(dt, initial_weights)` → optimised weights

4. Union with current holdings to include liquidation candidates (missing assets get weight `0.0`)

5. `OrderSizer(dt, full_weights)` → target share quantities

6. `broker.get_portfolio_as_dict(...)` → current share quantities

7. Delta calc (`target - current`) → incremental `list[Order]`

8. `ExecutionHandler` submits orders to `broker.submit_order(...)` (queued for later execution)

1.5 Broker execution and accounting path

For the detailed execution mechanics of `broker.update(dt)`—mark-to-market refresh, exchange-open gating, queue draining, bid/ask fill pricing, and fee calculation—see Section 1.3.1 above and Section 1.5.1 below.

At the accounting boundary, the key hand-off is:

`SimulatedBroker` → `Transaction(...)` → `Portfolio.transact_asset(...)`

From that point onward, the portfolio layer applies the fill and updates state.
Accounting side effects after a fill:

- Position state updates (buy/sell quantities, avg prices, realised/unrealised PnL).
- Cash debited/credited by consideration plus commission.
- Portfolio event history append for auditability.
- Portfolio-level values recomputed (`total_market_value`, `total_equity`, aggregate PnL).

1.5.1 Exchange gate and execution lifecycle details

- `Exchange` exposes one critical contract: `is_open_at_datetime(dt) -> bool`; broker execution is gated on this check.
- `SimulatedExchange` models weekday-only NYSE-style hours (`14:30 <= t < 21:00` UTC), and localises naive timestamps to UTC before comparison.
- Rebalance output (`list[Order]`) flows into `ExecutionHandler.__call__(dt, rebalance_orders)`, which first applies `ExecutionAlgorithm`, then submits orders.
- Default backtest execution uses `MarketOrderExecutionAlgorithm`, a pass-through implementation that leaves order lists unchanged.
- With `submit_orders=True` (default in backtests), the execution handler submits each order via `broker.submit_order(...)` and triggers `broker.update(dt)`.
- If the exchange is closed at that `dt`, orders remain queued and are executed on the next open-event update; this creates a clear submit-now/execute-later boundary.
- `submit_orders=False` is useful for dry-run analysis of portfolio construction and order generation without mutating broker state.

1.6 End products from `run()`

- `equity_curve`: chronological `(dt, total_equity)` points (after burn-in, typically at `market_close`).
- `target_allocations`: rebalance-time target allocations produced by portfolio construction.

Key data types at each boundary

Hand-off point	Type
`SimulationEngine` → main loop	`SimulationEvent(ts: pd.Timestamp, event_type: str)`
`AlphaModel` → `RiskModel` / optimiser	`dict[str, float]`
`PortfolioOptimiser` → `OrderSizer`	`dict[str, float]`
`OrderSizer` → rebalance diff	`dict[str, dict]` (target quantities)
`PortfolioConstructionModel` → `ExecutionHandler`	`list[Order]`
`ExecutionHandler` → `SimulatedBroker`	`Order(dt, asset, quantity)`
`SimulatedBroker` → `Portfolio`	`Transaction(asset, quantity, dt, price, commission)`
`broker.get_account_total_equity()` → equity curve	`float`

1.7 Sequence diagram (compact)

Timing note:

- In the default session, rebalance orders are usually generated/submitted at `market_close`, when the exchange is already closed.
- Those queued orders are then eligible to fill on the next `market_open broker.update(dt)` (subject to exchange-open check).

1.8 Portfolio construction flowchart (tiny)

Legend: `Current portfolio quantities` is sourced from `broker.get_portfolio_as_dict(...)`.

2 Strategy logic (using `TopNMomentumAlphaModel` as an example)

1. **Universe** — All SPDR US sector ETFs (`XLB`, `XLC`, `XLE`, `XLF`, `XLI`, `XLK`, `XLP`, `XLU`, `XLV`, `XLY`). Because `XLC` was not listed until June 2018, a `DynamicUniverse` is used so it is only eligible from that date onwards.
2. **Signal** — 126-business-day ($\approx$ 6-month) holding-period return momentum, calculated via `MomentumSignal`.
3. **Alpha Model (`TopNMomentumAlphaModel`)** — At each rebalance event, ranks every eligible asset by its momentum score and assigns an **equal weight of 1/N** to the top-N assets ($N = 3$ by default). All other assets receive a weight of 0.
4. **Rebalance** — `end_of_month` (the last trading day of every calendar month).
5. **Burn-in** — The first year of data (1999-01-01) is used purely for signal warm-up; performance statistics begin after that date.
6. **Benchmark** — A static 100% allocation to SPY using `FixedSignalsAlphaModel` with a `buy_and_hold` rebalance schedule.

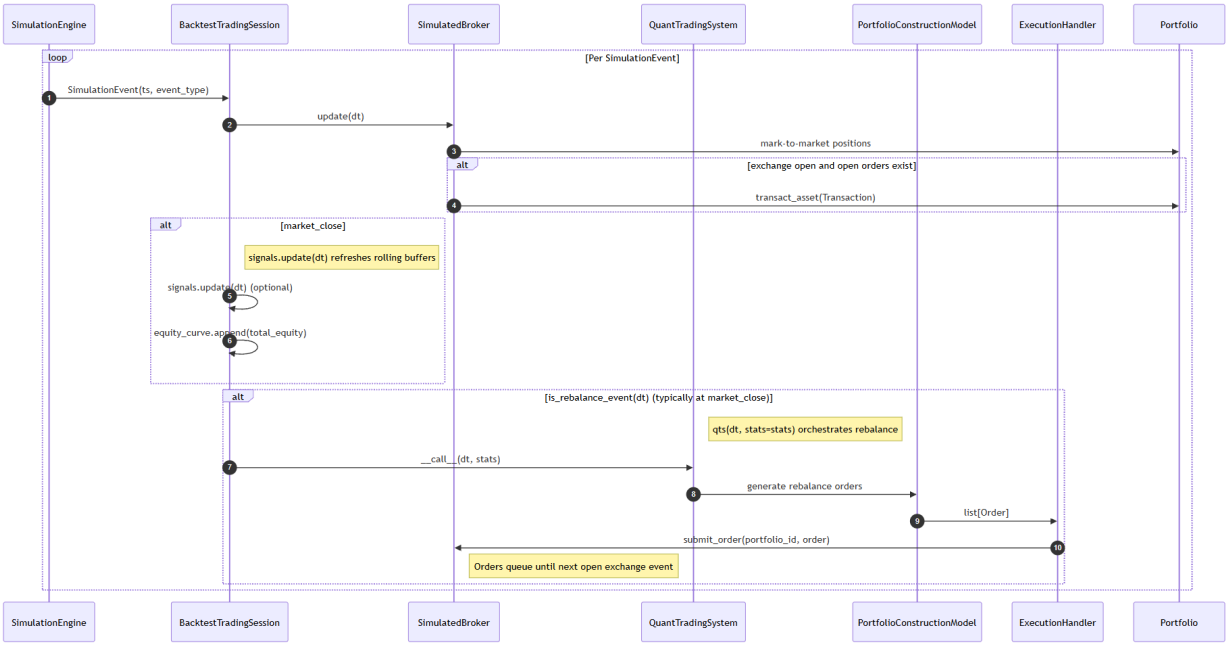

Figure 1: Compact sequence view of the backtest workflow.

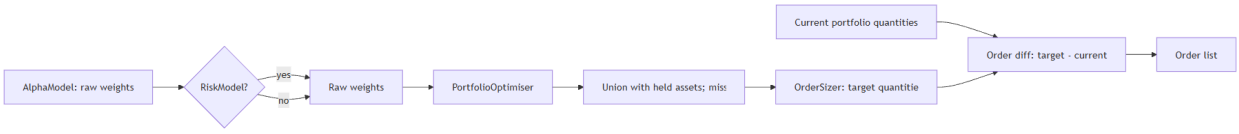

Figure 2: Portfolio construction flow from alpha weights to rebalance orders.